



Name: Deras Pangestu Gusti (08)
Class: SIB 1G
NIM: 254107060114

JOBSHEET XI LINKED LIST

1. Learning Objectives

After completing this practical session, students will be able to:

1. Construct a linked list data structure
2. Implement a linked list in a program
3. Identify problems that can be effectively solved using a linked list

2. Labs Activities

2.1 Experiment 1: Implementing Single Linked List

Allocated times: 30 minutes

In this experiment, we will practice how to create a Single Linked List using Node-based data representation, access the linked list, and implement data insertion methods.

1. In the project created for this semester, create a new package named **week12**.
2. Create the following classes:

- a. Student00.java
- b. Node00.java
- c. SingleLinkedList00.java
- d. SLLMain00.java

Modify 00 with your number

3. Implement **Student** following this class diagram:

Student
nim: String name: String className: String gpa: double
Student() Student (nm: String, name: String, kls: String, ip: double) print(): void

4. The following code is the Student class implementation

```
public class Student {  
    String nim, name, className;  
    double gpa;  
  
    public Student(){  
    }  
    public Student(String nm, String nama, String kls, double ip){
```



```

        nim = nm;
        name = nama;
        className = kls;
        gpa = ip;
    }
    void print(){
        System.out.println(nim+" - "+name+" - "+className+" - "+gpa);
    }
}

```

5. The following code snippet depicts the **Node** class implementation. Please make a note that the **Node** class manages the **Student** object data, that is why we create **Student** attribute within **Node** class. **You will need to modify it based on the data type of data that you manage.**

```

public class Node {
    Student data;
    Node next;
    public Node(){
    }
    public Node(Student data, Node next){
        this.data = data;
        this.next = next;
    }
}

```

6. Create a new class named **SingleLinkedList**
7. Add **head** attributes within **SingleLinkedList** class

```

public class SingleLinkedList {
    Node head;
    Node tail;
}

```

8. In the next step we will implement the basic method of single linked list
9. Create a new method, **isEmpty()**.

```

boolean isEmpty(){
    return (head==null);
}

```

10. Implement traversal process for displaying all data managed in the single linked list

```

void print(){
    if(!isEmpty()){
        Node tmp = head;
    }
}

```

```

        System.out.println("LinkedList Data:");
        while(tmp!=null){
            tmp.data.print();
            tmp = tmp.next;
        }
    }else{
        System.out.println("LinkedList is empty!!");
    }
}

```

11. Add **addFirst()** method as follows:

```

void addFirst(Student std){
    Node newNode = new Node(std, null);
    if(isEmpty()){
        head = newNode;
        tail = newNode;
    }else{
        newNode.next = head;
        head = newNode;
    }
}

```

12. Create a new method, **addLast()** as follows:

```

void addLast(Student std){
    Node newNode = new Node(std, null);
    if(isEmpty()){
        head = newNode;
        tail = newNode;
    }else{
        tail.next = newNode;
        tail = newNode;
    }
}

```

13. Add **insertAfter()**, to add a new student data after a **key** specified. In this example, we use student name as the key.

```

void insertAfter(Student std, String key){
    Node newNode = new Node(std, null);
    Node temp = head;
    do {
        if (temp.data.name.equalsIgnoreCase(key)) {
            newNode.next = temp.next;
            temp.next = newNode;
            if (newNode.next == null) {
                tail = newNode;
            }
        }
    }
}

```

```

        break;
    }
    temp = temp.next;
} while (temp != null);
}

```

14. Add these following codes to add a node based on defined index

```

public void insertAt(int index, Student std) {
    if (index < 0) {
        System.out.println("Wrong index!!");
    } else if (index == 0) {
        addFirst(std);
    } else {
        Node temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new Node(std, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}

```

15. Create a new **Main** class, with a **main** method inside. Then create a **SingleLinkedList** object.

```

public static void main(String[] args) {
    SingleLinkedList sll = new SingleLinkedList();
}

```

16. Add 4 student objects, **std1**, **std2**, **std3** and **std4**.

```

Student std1 = new Student("001", "Student 1", "TI-1I", 3.89);
Student std2 = new Student("002", "Student 2", "TI-1I", 3.45);
Student std3 = new Student("003", "Student 3", "TI-1I", 3.20);
Student std4 = new Student("004", "Student 4", "TI-1I", 3.00);

```

17. Add the students object into SingleLinkedList object, following these scenarios:

```

sll.print();
sll.addFirst(std4);
sll.print();
sll.addLast(std1);
sll.print();
sll.insertAfter(std3, "Student 4");
sll.insertAt(2, std2);
sll.print();

```

2.1.1 Output Verification

Observe the output of your program and compare it with the following output!

```
LinkedList is empty!!
LinkedList Data:
004 - Student 4 - TI-1I - 3.0
LinkedList Data:
004 - Student 4 - TI-1I - 3.0
001 - Student 1 - TI-1I - 3.89
LinkedList Data:
004 - Student 4 - TI-1I - 3.0
003 - Student 3 - TI-1I - 3.2
002 - Student 2 - TI-1I - 3.45
001 - Student 1 - TI-1I - 3.89
```

```
1  package com.jobsheet12;
2
3  public class Student08
4  {
5      String    nim,
6              name,
7              className;
8      double    gpa;
9
10     public Student08() {}
11     public Student08(String nm, String nama, String kls, double ip)
12     {
13         nim        = nm;
14         name        = nama;
15         className   = kls;
16         gpa         = ip;
17     }
18
19     void print()
20     {
21         System.out.println(nim + " - " + name + " - " + className + " - " + gpa);
22     }
23 }
```

```
1  package com.jobsheet12;
2
3  public class Node08
4  {
5      Student08 data;
6      Node08 next;
7
8      public Node08() {}
9      public Node08(Student08 data, Node08 next)
10     {
11         this.data = data;
12         this.next = next;
13     }
14 }
```

```
1  package com.jobsheet12;
2
3  public class SingleLinkedList08
4  {
5      Node08 head, tail;
6
7      boolean isEmpty()
8      {
9          return head == null;
10     }
11
12     void print()
13     {
14         if (!isEmpty())
15         {
16             Node08 tmp = head;
17             System.out.println(x: "LinkedList Data:");
18
19             while (tmp != null)
20             {
21                 tmp.data.print();
22                 tmp = tmp.next;
23             }
24         }
25         else
26             System.out.println(x: "LinkedList is empty!!");
27     }
28
29     void addFirst (Student08 std)
30     {
31         Node08 newNode = new Node08(std, next: null);
32
33         if (isEmpty())
34         {
35             head = newNode;
36             tail = newNode;
37         }
38         else
39         {
40             newNode.next = head;
41             head = newNode;
42         }
43     }
44 }
```



```
45 void addLast (Student08 std)
46 {
47     Node08 newNode = new Node08(std, next: null);
48
49     if (isEmpty())
50     {
51         head = newNode;
52         tail = newNode;
53     }
54     else
55     {
56         tail.next = newNode;
57         tail = newNode;
58     }
59 }
60
61 void insertAfter (Student08 std, String key)
62 {
63     Node08 newNode = new Node08(std, next: null);
64     Node08 temp = head;
65
66     do
67     {
68         if (temp.data.name.equalsIgnoreCase((key)))
69         {
70             newNode.next = temp.next;
71             temp.next = newNode;
72
73             if (newNode.next == null)
74                 tail = newNode;
75
76             break;
77         }
78
79         temp = temp.next;
80     } while (temp != null);
81 }
82
83
```

```
84 public void insertAt (int index, Student08 std)
85 {
86     if (index < 0)
87         System.out.println(x: "Wrong Index!!");
88     else if (index == 0)
89         addFirst(std);
90     else
91     {
92         Node08 temp = head;
93
94         for (int i = 0; i < index; i++)
95             temp = temp.next;
96
97         temp.next = new Node08(std, temp.next);
98
99         if (temp.next.next == null)
100             tail = temp.next;
101     }
102 }
103
```

```

1  package com.jobsheet12;
2
3  public class SLLMain08
4  {
5      Run main | Debug main | Run | Debug
6      public static void main(String[] args)
7      {
8          SingleLinkedList08 sll = new SingleLinkedList08();
9
10         Student08 std1 = new Student08(nm: "001", nama: "Student 1", kls: "TI-1I", ip: 3.89);
11         Student08 std2 = new Student08(nm: "002", nama: "Student 2", kls: "TI-1I", ip: 3.45);
12         Student08 std3 = new Student08(nm: "003", nama: "Student 3", kls: "TI-1I", ip: 3.20);
13         Student08 std4 = new Student08(nm: "004", nama: "Student 4", kls: "TI-1I", ip: 3.00);
14
15         sll.print();
16         sll.addFirst(std4);
17         sll.print();
18         sll.addLast(std1);
19         sll.print();
20         sll.insertAfter(std3, key: "Student 4");
21         sll.inserAt(index: 2, std2);
22         sll.print();
23     }
24 }

```

```

LinkedList is empty!!
LinkedList Data:
004 - Student 4 - TI-1I - 3.0
LinkedList Data:
004 - Student 4 - TI-1I - 3.0
001 - Student 1 - TI-1I - 3.89
LinkedList Data:
004 - Student 4 - TI-1I - 3.0
003 - Student 3 - TI-1I - 3.2
001 - Student 1 - TI-1I - 3.89
002 - Student 2 - TI-1I - 3.45

```

2.1.2 Questions!

1. Why does compiling the program code result in the message "Linked List is Empty" on the first line?

The message appears because, in the main method, the sll.print() method is invoked immediately after the SingleLinkedList object is instantiated. At this specific point in the execution, no student nodes have been added to the list yet. Consequently, the head attribute is still null, causing the isEmpty() check to return true and trigger the "LinkedList is empty!!" print statement.

2. Explain the general purpose of the variable temp in each method!
The variable temp (or tmp) serves as a temporary pointer or iterator used to traverse the linked list. Since a single linked list only provides a direct reference to the head, one must "walk" through the nodes by following the next references to reach a specific position or to display all data. Using temp allows the programme to navigate the list without losing the original reference to the head, which is vital for maintaining the list's structure.
3. Modify the code so that data can be added via keyboard input!


```

1  package com.jobsheet12;
2
3  import java.util.Scanner;
4
5  public class SLLMain08Modified
6  {
7      Run | Debug | Run main | Debug main
8      public static void main(String[] args)
9      {
10         Scanner sc = new Scanner(System.in);
11
12         SingleLinkedList08 sll = new SingleLinkedList08();
13
14         System.out.print(s: "Input NIM: ");
15         String nim = sc.nextLine();
16         System.out.print(s: "Input Name: ");
17         String name = sc.nextLine();
18         System.out.print(s: "Input Class: ");
19         String className = sc.nextLine();
20         System.out.print(s: "Input GPA: ");
21         double gpa = sc.nextDouble();
22
23         Student08 newStd = new Student08(nim, name, className, gpa);
24         sll.addLast(newStd);
25
26         Student08 std1 = new Student08(nm: "001", nama: "Student 1", kls: "TI-1I", ip: 3.89);
27         Student08 std2 = new Student08(nm: "002", nama: "Student 2", kls: "TI-1I", ip: 3.45);
28         Student08 std3 = new Student08(nm: "003", nama: "Student 3", kls: "TI-1I", ip: 3.89);
29         Student08 std4 = new Student08(nm: "004", nama: "Student 4", kls: "TI-1I", ip: 3.00);
30
31         sll.print();
32         sll.addFirst(std4);
33         sll.print();
34         sll.addLast(std1);
35         sll.print();
36         sll.insertAfter(std3, key: "Student 4");
37         sll.inserAt(index: 2, std2);
38         sll.print();
39     }
40 }

```

4. What would happen if we did not use the **tail** attribute? Would it affect the code implementation? Please explain.

Removing the tail attribute would not make the implementation impossible, but it would significantly decrease the efficiency of certain operations.

1. *Without a tail pointer: To perform an addLast() operation, the programme would have to start at the head and traverse the entire list until it reaches the final node. This changes the time complexity from $O(1)$ (constant time) to $O(n)$ (linear time).*
2. *With a tail pointer: The programme has an immediate reference to the end of the list, allowing for an instantaneous addition of new nodes to the rear.*

2.2 Experiment 2: Accessing Element in Single Linked List

Allocated time: 30 minutes

Teaching Team of Algorithm and Data Structure 2025-2026

Department of Information Technology-Politeknik Negeri Malang

In this practical session, we will learn how to access elements, retrieve their index, and perform data deletion in a Single Linked List.

2.2.1 Experiment Steps

1. Open **SingleLinkedList** class that is already created in Experiment 1.
2. Add a **getData()** method to get data at a specific index in the SingleLinkedList class.

```
Student getData(int idx){
    if(isEmpty()){
        System.out.println("LinkedList is empty!!");
        return null;
    }
    Node tmp = head;
    for(int i=0; i<idx;i++){
        tmp = tmp.next;
    }
    return tmp.data;
}
```

3. Add a new method to get the index of a data specified. This method is named `indexOf()`.

```
int indexOf(String key){
    if(isEmpty()){
        System.out.println("LinkedList is empty!!");
        return -1;
    }
    Node tmp = head;
    int idx = 0;
    while(tmp != null && !tmp.data.name.equalsIgnoreCase(key)){
        tmp = tmp.next;
        idx++;
    }
    if(tmp == null){
        return -1;
    }else{
        return idx;
    }
}
```

4. Add a new method named `removeFirst()` to remove the first element in single linked list object

```
void removeFirst(){
    if(isEmpty()){
        System.out.println("LinkedList is empty!!");
    }else if(head==tail){
        head = tail = null;
    }else{
        head = head.next;
    }
}
```

5. Add a method to remove the last element in the SingleLinkedList class, named `removeLast()`

```
void removeLast(){
    if(isEmpty()){
        System.out.println("LinkedList is empty!!");
    }else if(head==tail){
        head = tail = null;
    }else{
        Node tmp = head;
        while(tmp.next != tail){
            tmp = tmp.next;
        }
        tmp.next = null;
        tail = tmp;
    }
}
```

```
}
```

6. As the next step, the **remove()** method will be implemented.

```
public void remove(String key) {
    if (isEmpty()) {
        System.out.println("LinkedList is empty!!");
    } else {
        Node temp = head;
        while (temp != null) {
            if ((temp.data.name.equalsIgnoreCase(key)) && (temp ==
head)) {
                removeFirst();
                break;
            } else if (temp.next.data.name.equalsIgnoreCase(key)) {
                temp.next = temp.next.next;
                if (temp.next == null) {
                    tail = temp;
                }
                break;
            }
            temp = temp.next;
        }
    }
}
```

7. Implement a method to **removeAt()** a node by a specified index.

```
public void removeAt(int index) {
    if (index == 0) {
        removeFirst();
    } else {
        Node temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = temp.next.next;
        if (temp.next == null) {
            tail = temp;
        }
    }
}
```

8. Next, try accessing and deleting data in the **main** method of the **Main** class by adding the following code.

```
System.out.println("Data at index 1 is:");
Student data = sll.getData(1);
data.print();
```

```
int idx = sll.indexOf("Student 1");
System.out.println("Student 1 is located at index: "+idx);

sll.removeFirst();
sll.removeLast();
sll.print();
sll.removeAt(0);
sll.print();
```

9. Re-run the **Main** class

```
104      Student08 getData (int idx)
105      {
106          if (isEmpty())
107          {
108              System.out.println(x: "LinkedList is empty!!");
109              return null;
110          }
111
112          Node08 tmp = head;
113
114          for (int i = 0; i < idx; i++)
115              tmp = tmp.next;
116
117          return tmp.data;
118      }
119
120      int indexOf (String key)
121      {
122          if (isEmpty())
123          {
124              System.out.println(x: "LinkedList is empty!!");
125              return -1;
126          }
127
128          Node08 tmp = head;
129          int idx = 0;
130
131          while (tmp != null && !tmp.data.name.equalsIgnoreCase(key))
132          {
133              tmp = tmp.next;
134              idx++;
135          }
136
137          if (tmp == null)
138              return -1;
139          else
140              return idx;
141      }
142
```

```
143     void removeFirst()
144     {
145         if (isEmpty())
146             System.out.println(x: "LinkedList is empty!!");
147         else if (head == tail)
148             head = tail = null;
149         else
150             head = head.next;
151     }
152
153     void removeLast()
154     {
155         if (isEmpty())
156             System.out.println(x: "LinkedList is empty!!");
157         else if (head == tail)
158             head = tail = null;
159         else
160         {
161             Node08 tmp = head;
162
163             while (tmp.next != tail)
164                 tmp = tmp.next;
165
166             tmp.next = null;
167             tail = tmp;
168         }
169     }
170
```

```

171     public void remove (String key)
172     {
173         if (isEmpty())
174             System.out.println(x: "LinkedList is empty!!");
175         else
176         {
177             Node08 temp = head;
178
179             while (temp!= null)
180             {
181                 if ((temp.data.name.equalsIgnoreCase(key)) && (temp == head))
182                 {
183                     removeFirst();
184                     break;
185                 }
186                 else if (temp.next.data.name.equalsIgnoreCase(key))
187                 {
188                     temp.next = temp.next.next;
189
190                     if (temp.next == null)
191                         tail = temp;
192                     break;
193                 }
194             }
195
196             temp = temp.next;
197         }
198     }
199 }
200
201 public void removeAt (int index)
202 {
203     if (index == 0)
204         removeFirst();
205     else
206     {
207         Node08 temp = head;
208
209         for (int i = 0; i < index; i++)
210             temp = temp.next;
211
212         temp.next = temp.next.next;
213
214         if (temp.next == null)
215             tail = temp;
216     }
217 }
218
219 }
220
221
222
223     System.out.println(x: "Data at index 1 is: ");
224     Student08 data = sll.getData(idx: 1);
225     data.print();
226
227     int idx = sll.indexOf(key: "Student 1");
228     System.out.println("Student 1 is located at index: " + idx);
229
230     sll.removeFirst();
231     sll.removeLast();
232     sll.print();
233     sll.removeAt(index: 0);
234     sll.print();
235 }
236 }

```



```
Data at index 1 is:
003 - Student 3 - TI-1I - 3.2
Student 1 is located at index: 2
LinkedList Data:
003 - Student 3 - TI-1I - 3.2
001 - Student 1 - TI-1I - 3.89
LinkedList Data:
001 - Student 1 - TI-1I - 3.89
```

2.2.2 Output verification

Observe the output of your program and compare it with the following output!

```
Data at index 1 is:
003 - Student 3 - TI-1I - 3.2
Student 1 is located at index: 3
LinkedList Data:
003 - Student 3 - TI-1I - 3.2
002 - Student 2 - TI-1I - 3.45
LinkedList Data:
002 - Student 2 - TI-1I - 3.45
```

2.2.3 Questions

1. Why is the **break** keyword used in the remove function? Explain!

break stops the loop immediately after a successful removal so the method does not continue scanning the list.

2. Explain the purpose of the code below in the remove method.

```
temp.next = temp.next.next;
if (temp.next == null) {
    tail = temp;
}
```

- a. *temp.next = temp.next.next; → this reassigns the next reference of temp so that it skips over the node to be removed; effectively the node after temp is unlinked from the list and becomes eligible for garbage collection.*
- b. *if (temp.next == null) { tail = temp; } → after unlinking, if temp.next is null it means the removed node was the last node; therefore temp is now the new last node and tail must be updated to point to it. This keeps the tail reference consistent with the list's actual end.*

3. Assignments

Allocated times: 50 menit

Create a queue-based program for student service operations with the following requirements:

- a. Implement the queue using a **Linked List-based Queue**.
- b. The program should be a **new project**, not a modification of an existing example.
- c. When a student wants to join the queue, they must **register their information**.
- d. Include functions to **check if the queue is empty**, **check if it is full**, and **clear the queue**.
- e. Implement **adding a student to the queue**.
- f. Implement **calling the next student in the queue**.
- g. Display the **first (front)** and **last (rear)** student in the queue.
- h. Display the **total number of students** still in the queue.


```
1  package com.jobsheet12;
2
3  public class StudentData
4  {
5      String nim,
6          name,
7          className;
8      double gpa;
9
10     public StudentData (String nm, String nama, String kls, double ip)
11     {
12         nim      = nm;
13         name     = nama;
14         className = kls;
15         gpa      = ip;
16     }
17
18     void print()
19     {
20         System.out.println(nim + " - " + name + " - " + className + " - " + gpa);
21     }
22 }

```

```
1  package com.jobsheet12;
2
3  public class NodeStudentData
4  {
5      StudentData data;
6      NodeStudentData next;
7
8      public NodeStudentData (StudentData data, NodeStudentData next)
9      {
10         this.data = data;
11         this.next = next;
12     }
13 }

```

```
1  package com.jobsheet12;
2
3  public class LinkedListStudentQueue
4  {
5      NodeStudentData head, tail;
6      int size;
7
8      boolean isEmpty()
9      {
10         return head == null;
11     }
12
13     void printEmpty()
14     {
15         System.out.println(x: "Student Data is Empty!!");
16     }
17
18     String isFull()
19     {
20         return "LinkedList is full!!";
21     }
22
23     void addStudent (StudentData std)
24     {
25         NodeStudentData newNode = new NodeStudentData(std, next: null);
26
27         if (isEmpty())
28         {
29             head = newNode;
30             tail = newNode;
31         }
32         else
33         {
34             tail.next = newNode;
35             tail = newNode;
36         }
37
38         size++;
39     }
40 }
```

```
41  StudentData removeStudent()
42  {
43      if (isEmpty())
44      {
45          printEmpty();
46          return null;
47      }
48
49      StudentData remove = head.data;
50      head = head.next;
51
52      if (head == null)
53          tail = null;
54
55      size--;
56      return remove;
57  }
58
59  StudentData printHead()
60  {
61      if (isEmpty())
62      {
63          printEmpty();
64          return null;
65      }
66      else
67          return head.data;
68  }
69
70  StudentData printTail()
71  {
72      if (isEmpty())
73      {
74          printEmpty();
75          return null;
76      }
77      else
78          return tail.data;
79  }
80
81  int printSize()
82  {
83      return size;
84  }
85
86  void clear()
87  {
88      head = tail = null;
89      size = 0;
90  }
91 }
```

```
1  package com.jobsheet12;
2
3  import java.util.Scanner;
4
5  public class StudentDataMain
6  {
7      static void menu()
8      {
9          System.out.println(x: "\n=== Queue Menu ===");
10         System.out.println(x: "1. Register (add) a students");
11         System.out.println(x: "2. Call (remove) the students");
12         System.out.println(x: "3. See Front (head) & Rear (tail)");
13         System.out.println(x: "4. Total number of students in the queue");
14         System.out.println(x: "5. Clear the queue");
15         System.out.println(x: "0. Exit");
16         System.out.print(s: "choose: ");
17     }
18
19     Run main | Debug main | Run | Debug
20     public static void main(String[] args)
21     {
22         Scanner sc = new Scanner(System.in);
23
24         LinkedListStudentQueue sll = new LinkedListStudentQueue();
25
26         int option;
27         do
28         {
29             menu();
30             option = sc.nextInt(); sc.nextLine();
```



```
31      switch (option)
32      {
33          case 1:
34              System.out.print(s: "NIM   : ");
35              String nim   = sc.nextLine();
36              System.out.print(s: "Name : ");
37              String name  = sc.nextLine();
38              System.out.print(s: "Class : ");
39              String studentClass = sc.nextLine();
40              System.out.print(s: "GPA   : ");
41              double gpa = sc.nextDouble(); sc.nextLine();
42
43              sll.addStudent(new StudentData(nim, name, studentClass, gpa));
44              System.out.println(x: "The Student has been added in Queue");
45
46              break;
47
48          case 2:
49              if (sll.isEmpty())
50                  sll.printEmpty();
51              else
52              {
53                  StudentData remove = sll.removeStudent();
54                  System.out.printf(format: "%s has been removed!\n", remove.name);
55              }
56              break;
57
58          case 3:
59              if (sll.isEmpty())
60                  sll.printEmpty();
61              else
62              {
63                  StudentData front = sll.printHead();
64                  StudentData rear  = sll.printTail();
65
66                  System.out.print(s: "Front : ");
67                  front.print();
68                  System.out.print(s: "Rear   : ");
69                  rear.print();
70              }
71
72              break;
73      }
```

```
74         case 4:
75             System.out.println("Total : " + sll.printSize());
76             break;
77
78         case 5:
79             sll.clear();
80             System.out.println(x: "The Queue has been cleared.");
81             break;
82
83         case 0:
84             System.out.println(x: "Thanks!!");
85             break;
86
87         default:
88             System.out.println(x: "Input Invalid!! Please, Try Again!");
89     }
90 }
91 while (option != 0);
92
93 sc.close();
94 }
95 }
```

```
=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 2
Student Data is Empty!!
```

```
=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 3
Student Data is Empty!!
```

```
=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 4
Total : 0
```

```
=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 1
NIM : 001
Name : Deras
Class : 1G
GPA : 3,52
The Student has been added in Queue

=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 1
NIM : 002
Name : Gurami
Class : 1G
GPA : 3,77
The Student has been added in Queue

=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 1
NIM : 003
Name : Dadar
Class : 1G
GPA : 3,16
The Student has been added in Queue
```

```
=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 3
Front : 001 - Deras - 1G - 3.52
Rear  : 003 - Dadar - 1G - 3.16

=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 4
Total : 3

=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 2
Deras has been removed!
=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 4
Total : 2
```



```
=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 5
The Queue has been cleared.

=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 4
Total : 0

=== Queue Menu ===
1. Register (add) a students
2. Call (remove) the students
3. See Front (head) & Rear (tail)
4. Total number of students in the queue
5. Clear the queue
0. Exit
choose: 0
Thanks!!
```

Github:

<https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data/tree/main/src/com/jobsheet12>